Ch 04: Performing Calculations with Loops

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Calculating a Running Total

Calculating a Running Total

A running total is a sum of numbers that accumulates with each iteration of a loop. The variable used to keep the running total is called an accumulator.

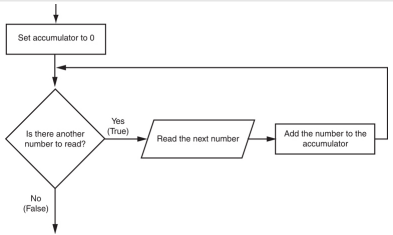


Figure 1: Logic for calculating a running total (Gaddis, 2021, pg.211)

- Programs often need to calculate a total of a series of numbers
  - Typically include two elements
    1. A loop that reads each number in series
    2. An **accumulator** variable
  - Known as program that keeps a running total: accumulates total and reads in series
  - At end of loop, accumulator will hold the total

EAST TEXAS A&M

The Augmented Assignment Operators

- In many assignment statements, the variable on the left side of the = operator also appears on the right side
- **Augmented assignment operators**: special set of operators designed for this type of common assignment

Table 1: Augmented assignment operators

Operator	Example Usage	Equivalent to
`+=`	`x += 5`	`x = x + 5`
`-=`	`y -= 2`	`y = y - 2`
`*=`	`z *= 10`	`z = z * 2`
`/=`	`a /= b`	`a = a / b`
`%=`	`c %= 3`	`c = c % 3`
`//=`	`x //= 3`	`x = x // 3`
`**=`	`y **= 2`	`y = y**2`

EAST TEXAS A&M

Sentinels

- **Sentinel**: special value that marks the end of a sequence of items
  - When a program reaches a sentinel, it knows that the end of the sequence of items was reached, and the loop terminates
  - Must be distinctive enough so as not to be mistaken for a regular value in the sequence
  - Example: when reading an input file, empty line can be used as a sentinel

Sentinel

A sentinel is a special value that marks the end of a sequence of values



EAST TEXAS A&M

Input Validation Loops

- Computer cannot tell the difference between good and bad data
  - If user provides bad input, program will produce bad output
  - It is important to design programs such that bad input is never accepted
- **Input validation**: inspecting input before it is processed by the program
  - If input is invalid, prompt user to enter correct data
  - Commonly accomplished using a `while` loop which repeats as long as the input is bad
    - If input is bad, display error message and try again
    - If input is good, continue past loop and process input

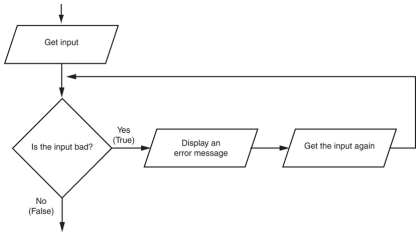


Figure 2: Logic for an input validation loop (Gaddis, 2021, pg.218)

EAST TEXAS A&M

Nested Loops

- **Nested loop**: loop that is contained inside another loop
  - Example: analog clock works like nested loop
    - Hours hand moves once for every twelve movements of the minutes hand
    - Seconds hand moves 60 times for each movement of the minutes hand
- Key points about nested loops:
  - Inner loop goes through all of its iterations for each iteration of outer loop
  - Inner loops complete their iterations faster than outer loops
- Total number of iterations in nested loop

$$\text{num iter inner loop} \times \text{num iter outer loop}$$

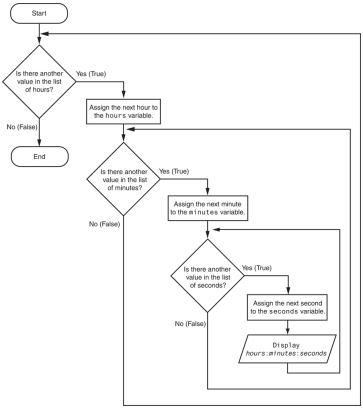


Figure 3: Flowchart for a clock simulator (Gaddis, 2021, pg.223)

EAST TEXAS A&M

Summary

- There are several common tasks or calculation you perform with a loop
  - Iterate over a series of items in a collection to process each item
  - For example, calculate a running total of items, using an **accumulator**
- Augmented assignment operators are common when updating the value of a variable, e.g. reassigning updated value back into original variable
- Many loops are *sentinel-controlled loops* that keep processing input until a special **sentinel** value is detected in the input.
- Another common task for loops is **input validation** where input is entered and checked to see if valid, and the loop does not continue until valid input has been received.
- Loops can be nested for many purposes



References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

EAST TEXAS A&M